

TaskEasy App

Complete Logic Documentation

Modules • Buttons • Workflows • Sheets • Backend Functions • Gaps • UAT

Prepared from uploaded screenshots, Index HTML, and Code.gs | 15 Jun 2026

- Purpose: make the current Apps Script + Google Sheets application understandable enough for refactoring, migration, QA, and future feature work.
- Scope: dashboard, delegation, work requests, checklist/KRA, FMS, MIS, reports, user/project admin, hierarchy, notifications, exports, cache and validation.
- Important note: this document separates current observed behavior from recommended final logic where gaps exist.

1. Executive Summary

TaskEasy is a role-based workflow management application built with Google Apps Script as backend, Google Sheets as database, and a single-page HTML frontend. The screenshots show a professional admin dashboard with modules for tasks, approvals, MIS, FMS, reports, users and hierarchy. The uploaded code confirms role-based lazy loading, client-side filtering/export, cached initial load, attachment upload to Drive, and multiple backend sheet operations.

- Main value: centralize daily task assignment, work request tracking, checklist/KRA completion, FMS flow execution and employee performance scoring.
- Core workflow: Create/Assign -> Pending -> Doer submits -> Approver approves or reworks -> Dashboard/MIS/Reports update.
- Primary risk: UI is strong, but backend needs stricter role enforcement, duplicate function cleanup and explicit FMS/Checklist generation standards before production scale.

2. Technology and Runtime Architecture

Layer	Current Implementation	Responsibility
Frontend	Index HTML + Tailwind + jQuery + DataTables + Select2 + SweetAlert2 + Chart.js	SPA rendering, forms, filters, tables, charts, exports and user interactions.
Backend	Google Apps Script Code.gs	Authentication, sheet CRUD, Drive uploads, email notifications, dashboard/MIS/report calculations.
Database	Google Sheets	Stores users, hierarchy, projects, transactional tasks, FMS and MIS history.
File Storage	Google Drive folder via FOLDER_ID	Stores attachments from delegation, work request and checklist submissions.
Cache	Browser localStorage + Apps Script CacheService	Speeds up initial app load and repeated calculations.

User -> HTML SPA -> google.script.run -> Code.gs -> Google Sheets / Drive / Mail -> HTML rerender

3. Roles and Access Matrix

Role	Allowed Scope	Main Screens	Important Rule
Super Admin	All company users and all data	All modules including Admin Settings and Set Hierarchy	Can manage users, projects, hierarchy, reports and all team data.
Admin	Self + mapped employees from Hierarchy Setup	Dashboard, Delegation, Work Request, Checklist, FMS, Approve/Review, MIS, Reports	Must not see employees outside assigned hierarchy.
Employee	Own data only	Dashboard, Delegation, Checklist, Work Request, FMS, Track Status/My Approvals	Can create/complete own workflow actions only; cannot manage users/projects/hierarchy.

Role Resolution Logic

Login -> checkCredentials(userId, password) -> return userName + role -> initializeApp() -> show/hide nav -> fetch data scoped by role

4. Sheet/Data Model Documentation

Sheet	Purpose
Admin Detail	Admin/Super Admin and user auth rows
Employee Detail	Employee auth/user rows
Hierarchy Setup	Admin to employees mapping
Projects	Project master with Active/Paused status
DELEGATED WORK	Delegation task transactions
WORK_REQUESTS	Work request transactions
CHECKLIST WORK	Checklist master/KRA setup
CHECKLIST TASK	Generated checklist instances
FMS	Flow Management System tasks from imported flow sheet
MIS History	Weekly MIS snapshot and target history

Admin Detail / Employee Detail

#	Column	Logic Meaning
1	User	Required for module logic / reporting / status calculation
2	Number	Required for module logic / reporting / status calculation
3	User ID	Required for module logic / reporting / status calculation
4	Email	Required for module logic / reporting / status calculation
5	Password	Required for module logic / reporting / status calculation
6	Role	Required for module logic / reporting / status calculation
7	Photo URL	Required for module logic / reporting / status calculation
8	Status	Required for module logic / reporting / status calculation

DELEGATED WORK

#	Column	Logic Meaning
1	Timestamp	Required for module logic / reporting / status calculation
2	Task ID	Required for module logic / reporting / status calculation
3	Delegated By	Required for module logic / reporting / status calculation
4	Delegated To	Required for module logic / reporting / status calculation
5	Description	Required for module logic / reporting / status calculation
6	Project	Required for module logic / reporting / status calculation
7	Target Date	Required for module logic / reporting / status calculation
8	Attachment	Required for module logic / reporting / status calculation
9	Task Status	Required for module logic / reporting / status calculation
10	Revision Date	Required for module logic / reporting / status calculation

11	Attachment by Doer	Required for module logic / reporting / status calculation
12	Final Remarks by Doer	Required for module logic / reporting / status calculation
13	Action Date	Required for module logic / reporting / status calculation
14	Rework Remark	Required for module logic / reporting / status calculation
15	Rating	Required for module logic / reporting / status calculation
16	Final Remarks	Required for module logic / reporting / status calculation
17	Approval Date	Required for module logic / reporting / status calculation
18	Total Delays	Required for module logic / reporting / status calculation
19	Status 2 / On Time Status	Required for module logic / reporting / status calculation
20	Priority	Required for module logic / reporting / status calculation
21	Email	Required for module logic / reporting / status calculation

WORK_REQUESTS

#	Column	Logic Meaning
1	Timestamp	Required for module logic / reporting / status calculation
2	Request ID	Required for module logic / reporting / status calculation
3	Requested By	Required for module logic / reporting / status calculation
4	Request For	Required for module logic / reporting / status calculation
5	Description	Required for module logic / reporting / status calculation
6	Project	Required for module logic / reporting / status calculation
7	Deadline	Required for module logic / reporting / status calculation
8	Attachment	Required for module logic / reporting / status calculation
9	Status	Required for module logic / reporting / status calculation
10	Remarks by Doer	Required for module logic / reporting / status calculation
11	Attachment by Doer	Required for module logic / reporting / status calculation
12	Final Remarks	Required for module logic / reporting / status calculation
13	Completion Date	Required for module logic / reporting / status calculation
14	Delay Days	Required for module logic / reporting / status calculation
15	On Time Status	Required for module logic / reporting / status calculation

CHECKLIST WORK

#	Column	Logic Meaning
1	Timestamp	Required for module logic / reporting / status calculation
2	Task ID	Required for module logic / reporting / status calculation
3	Delegated By	Required for module logic / reporting / status calculation
4	Delegated To	Required for module logic / reporting / status calculation
5	Task Description	Required for module logic / reporting / status calculation
6	Project	Required for module logic / reporting / status calculation
7	Task Frequency	Required for module logic / reporting / status calculation
8	Start Date	Required for module logic / reporting / status calculation
9	Day/Date	Required for module logic / reporting / status calculation
10	Attachment Required	Required for module logic / reporting / status calculation

CHECKLIST TASK

#	Column	Logic Meaning
1	Task ID	Required for module logic / reporting / status calculation
2	User Name	Required for module logic / reporting / status calculation
3	Plan Date	Required for module logic / reporting / status calculation
4	Description	Required for module logic / reporting / status calculation
5	Frequency	Required for module logic / reporting / status calculation
6	Project	Required for module logic / reporting / status calculation
7	Actual Date	Required for module logic / reporting / status calculation
8	Attachment	Required for module logic / reporting / status calculation
9	Remarks	Required for module logic / reporting / status calculation
10	Attachment Required	Required for module logic / reporting / status calculation
11	Delegated By	Required for module logic / reporting / status calculation
12	Total Delay	Required for module logic / reporting / status calculation
13	Status	Required for module logic / reporting / status calculation
14	Approval Status	Required for module logic / reporting / status calculation

Projects

#	Column	Logic Meaning
1	Project Name	Required for module logic / reporting / status calculation
2	Status	Required for module logic / reporting / status calculation

FMS

#	Column	Logic Meaning
1	Person ID	Required for module logic / reporting / status calculation
2	What	Required for module logic / reporting / status calculation
3	When	Required for module logic / reporting / status calculation
4	How	Required for module logic / reporting / status calculation
5	Who	Required for module logic / reporting / status calculation
6	FMS Name	Required for module logic / reporting / status calculation
7	Task Name	Required for module logic / reporting / status calculation
8	Step No	Required for module logic / reporting / status calculation
9	Planned Date	Required for module logic / reporting / status calculation
10	Actual Date	Required for module logic / reporting / status calculation
11	Form Link	Required for module logic / reporting / status calculation
12	Delay Days	Required for module logic / reporting / status calculation
13	On Time Status	Required for module logic / reporting / status calculation

MIS History

#	Column	Logic Meaning
1	Timestamp	Required for module logic / reporting / status calculation
2	Week ID	Required for module logic / reporting / status calculation
3	Employee	Required for module logic / reporting / status calculation
4	Category	Required for module logic / reporting / status calculation
5	KPI	Required for module logic / reporting / status calculation
6	Score	Required for module logic / reporting / status calculation
7	Total	Required for module logic / reporting / status calculation
8	Done	Required for module logic / reporting / status calculation

9	Pending	Required for module logic / reporting / status calculation
10	Late/Rework	Required for module logic / reporting / status calculation
11	On Time	Required for module logic / reporting / status calculation
12	Next Week Target	Required for module logic / reporting / status calculation

5. Application Boot and Session Logic

1. Login page captures User ID, Password and Remember Me flag.
2. Frontend calls checkCredentials(userId, password).
3. Backend finds matching user in cached Admin Detail / Employee Detail data.
4. If password matches and Status is Active, backend returns userName and role.
5. Frontend saves user/role in localStorage if Remember Me is checked, otherwise sessionStorage.
6. initializeApp() sets currentUser/currentUserRole, updates greeting, shows role-based navigation, loads cached SUPER_APP_CACHE if valid, otherwise calls getUnifiedAppData().
7. applyAppData() stores users/projects/dashboard/rawData/notifications in GLOBAL_STORE, updates sidebar notification dots, shows app view and opens dashboard.

Final Session Rules

- Never allow inactive users to log in.
- Never trust frontend role hiding as security; backend must check role and team scope on every action.
- Logout must clear localStorage/sessionStorage and cached app data.
- A cache hit must be tied to the same user, otherwise ignore cache.

6. Navigation and Lazy Loading Logic

Module	View ID	Visible To	Purpose
Dashboard	dashboard-view	All roles	Landing analytics module with role-based metrics, project/FMS status, trend chart, priority lists, and drill-down tables.
Checklists	checklist-view	All roles	Admin assigns recurring checklist/KRA tasks. Employees complete planned task instances.
Delegation	delegation-view	All roles	Admin/Super Admin assigns work to users. Doer submits proof. Approver completes or reworks.
Work Request	work-request-view	All roles	Any allowed user raises request for another user. Doer completes and requester/admin approves.
FMS System	fms-view	All roles	Flow-based tasks imported from FMS sheet and completed by assignee.
Approve/Review	approve-task-view	Admin/Super Admin; Employee as Track Status/My	Unified approval queue for delegation/work

		Approvals	request/checklist submissions.
MIS	mis-view	Admin/Super Admin	Employee performance scoring by pending, delayed, on-time, rework, and category KPIs.
Reports	reports-view	Admin/Super Admin	Exportable detailed tables for performance, delegation, work request, checklist, and project summaries.
Admin Settings	user-management-view	Super Admin	User and project management.
Set Hierarchy	hierarchy-view	Super Admin	Admin-to-employee team mapping for access control and dashboards.

setView(viewName): hide all .content-view -> show selected view -> if empty, call role-specific loader -> keep dashboard special cache handling

7. Complete Button and Action Matrix

Module/Area	Button or Control	Expected Working Logic	Main Function
Global/Header	Sidebar Toggle	Open/close sidebar. On mobile it also controls overlay. On desktop it collapses/expands layout.	UI only
Global/Header	Theme Toggle	Toggle html.dark, switch sun/moon icons, save theme in localStorage.	No server call
Global/Header	Refresh	Clear SUPER_APP_CACHE, reset GLOBAL_STORE, call getUnifiedAppData(), rerender active dashboard/module.	getUnifiedAppData
Global/Header	Profile	Toggle signed-in user dropdown.	UI only
Global/Header	Logout	Clear user/role storage and cache, hide app, show login.	No server call
Global/Common Table	Excel	Clean HTML then export visible table data to .xlsx.	Client XLSX
Global/Common Table	PDF	Clean HTML then export visible table data to PDF using jsPDF autoTable.	Client jsPDF
Global/Common Filter	X Clear	Reset filters, dates and period to All, then reload current view.	Module data call
Global/Common Filter	All/Today/This Week/Last Week/This Month/Last Month	Set period filter and reload module data.	Module data call
Dashboard	Team View	Set employee filter to All Team and reload dashboard metrics.	getDashboardPageData
Dashboard	My View	Set employee filter to current user and reload dashboard metrics.	getDashboardPageData
Dashboard	Metric Total/Done/Pending/Delayed click	Fetch and show filtered drill-down table for selected card/module/status.	getFilteredDataForCard

Dashboard	Critical Team Task chips	Filter critical team task cards by All, Work Request, Checklist, FMS, Delegation.	Client filter
Delegation	Add Task	Append a new delegation task card.	Client only
Delegation	Remove X	Remove task card; one card must remain.	Client only
Delegation	Send All Tasks	Validate rows, read attachments, save one row per doer, send email.	saveTask
Delegation	Done	Doer submits remarks/proof for approval.	submitTaskWrapper -> submitTaskForApproval
Work Request	Add Request	Append a new request card.	Client only
Work Request	Remove X	Remove request card; one card must remain.	Client only
Work Request	Submit All	Validate rows, parse date/time safely, read attachments, save rows, send email.	saveWorkRequest
Work Request	Done	Doer submits completion remarks/proof.	submitTaskWrapper -> updateWorkRequestStatus(Send for Approval)
Checklist	Add Task	Append a checklist item row with frequency, start date/time and attachment-required option.	Client only
Checklist	Assign All	Save checklist master rows for selected employees/project/frequency.	saveChecklistTask
Checklist	Done	Mark one planned checklist instance complete with remarks/proof.	markChecklistTaskDone
Checklist	Select All / Mark Selected Done	Bulk complete selected planned checklist instances.	markChecklistTasksDoneBulk
Approve/Review	Approve	Route by item type and update status to Completed/Approved.	updateStatusWrapper
Approve/Review	Rework	Route by item type and set status to Rework with remarks.	updateStatusWrapper
FMS	Open Form	Open FMS row form link in iframe modal.	Client only
FMS	Open in New Tab	Open iframe/form link in browser tab.	Client only
FMS	Done	Write Actual Date, calculate Delay Days and On Time Status.	markFmsTaskDone
FMS	Import Sheet	Should import/refresh FMS flow rows. UI exists in screenshot; backend must be added/verified.	Recommended: importFmsSheet
FMS	Analytics	Should open FMS analytics. UI exists in screenshot; backend/view must be added/verified.	Recommended: getFmsAnalytics
FMS	Generate with AI	Should create draft flow from prompt/sheet. UI exists in screenshot; backend must be added/verified.	Recommended: generateFmsDraft

MIS	Update	Save weekly target/snapshot for selected employee.	saveUserWeeklyScore / saveMisWeeklySnapshot
MIS	KPI Row Click	Open drill-down table showing tasks causing the selected negative KPI.	getDetailedDataForUser
Reports	Report Tabs	Load Performance/Delegation/Work Request/Checklist/Project report.	loadReportData -> getAllReportData/getEmployeePerformanceReport
Admin Settings	Add User	Open user modal and save to Admin Detail or Employee Detail by role.	upsertUser
Admin Settings	Edit User	Open modal with existing user and update row.	upsertUser
Admin Settings	Delete User	Confirm and delete user row.	deleteUser
Admin Settings	All Roles Filter	Filter user table by role.	Client DataTable filter
Admin Settings	Add Project	Prompt for project name and create Active project.	manageProject(add)
Admin Settings	Project Power Icon	Toggle Active/Paused status.	manageProject(toggleStatus)
Admin Settings	Project Delete	Delete project row.	manageProject(delete)
Set Hierarchy	Add Group	Open hierarchy modal for admin + employee members.	saveHierarchy
Set Hierarchy	Edit	Open existing group and update mapping.	saveHierarchy
Set Hierarchy	Delete	Remove hierarchy group and save remaining mappings.	saveHierarchy

8. Common UI Components Logic

8.1 Loader

- showLoading(true) displays overlay and starts visible seconds counter.
- showLoading(false) hides overlay and clears counter.
- For large tables, use local loader instead of full-screen loader when user can keep context.

8.2 DataTables

- initializeDataTable() renders exports, responsive table, sorting, search, pagination and performance options.
- Excel export should export cleaned visible table content, not raw HTML tags.
- PDF export should include report name/date, filtered table data and readable columns.
- Empty arrays must show empty state, not a blank card.

8.3 Filters

- Filters support employee, from user, project, status, dateFrom/dateTo and period chips.
- Period chips are All, Today, This Week, Last Week, This Month, Last Month.
- If dateFrom/dateTo exists and period is All, final filter should become Custom.
- Frontend filters can be used for already-loaded team tables, but backend filters are preferred for large datasets.

8.4 Attachments

- Frontend reads files as base64 with FileReader.
- Backend saveAttachmentToDrive(fileData, taskId) stores file in Drive folder and returns URL.
- Sheet stores multiple URLs separated by new lines.
- formatAttachments() should display View(n) button; viewAttachments() opens clickable list.

9. Dashboard Logic Documentation

The dashboard is the central control room. It is role-aware and uses current filter state to show Team View or My View. It includes summary metric cards, drill-down, project status, FMS status, trend chart, personal priority and critical team tasks.

Dashboard Metrics

Metric	Definition	Displayed In
Total	All tasks in scope for selected module/category/filter.	Delegation, Work Request, Checklist, FMS
Done	Completed tasks or rows with actual/completion date.	All summary cards
Pending	Not completed; includes Rework where doer action is required.	All summary cards
Delayed	Completed after plan/deadline or still pending after due date.	All summary cards and MIS

Dashboard Drill-down Logic

Metric click -> showFilteredData(cardType, status, value) -> getFilteredDataForCard(cardType, status, currentUser, role, filters) -> render detail table -> closeDetailView() restores lower dashboard

Priority Logic

- Personal priority: pending work assigned to logged-in user, sorted by overdue/nearest due date/high priority.
- Critical team tasks: Admin/Super Admin list of overdue or high-risk tasks grouped by assigned person.
- Critical chips allow filtering by module type without server call if all critical rows are already loaded.

10. Delegation Module Logic Documentation

Create / Assign Flow

Admin -> Delegation -> Assign Task -> select Delegate To + Project -> enter Task Detail + Date + Time + Priority + Attachments -> Send All Tasks -> saveTask() -> row(s) in DELEGATED WORK with Pending -> email doer

Employee Done Flow

Doer opens Pending Tasks -> Done -> remarks + proof -> submitTaskWrapper("Task") -> submitTaskForApproval() -> status Send for Approval -> Approve/Review queue

Approver Flow

Approve -> updateStatusWrapper("Delegation", id, "Completed") -> updateTaskStatus() -> approval date + final remarks + delay/on-time
Rework -> updateStatusWrapper("Delegation", id, "Rework") -> updateTaskStatus() -> rework remark -> doer sees again

Delegation Status Machine

Pending -> Send for Approval -> Completed
Pending -> Send for Approval -> Rework -> Send for Approval -> Completed

Delegation Validations

- Delegate To required and can be one or multiple users.
- Project required and must be active.
- Task detail required.
- Target date/time required; use ISO-like format internally.
- At least one task card must remain.

- Attachments optional unless company rule says required for high priority work.

11. Work Request Module Logic Documentation

Create Flow

Requester -> Create Request -> Request For (Doer) + Project + Description + Deadline Date + Deadline Time + Attachments -> Submit All -> saveWorkRequest() -> WORK_REQUESTS row with Pending -> email doer

Completion and Approval Flow

Doer -> Pending For Me -> Done -> remarks/proof -> updateWorkRequestStatus(Send for Approval) -> requester/admin approval queue
 Approve -> Completed
 Rework -> Rework + remarks -> doer sees again

Work Request Status Machine

Pending -> Send for Approval -> Completed
 Pending -> Send for Approval -> Rework -> Send for Approval -> Completed

Special Date Rule

- Frontend should use native input[type=date] and input[type=time].
- Frontend getSafeIsoDateTime() must send YYYY-MM-DDTHH:mm:ss.
- Backend parseWorkRequestDeadline_0 must parse manually and never rely on ambiguous new Date("07/05/2026").

12. Checklist and KRA Module Logic Documentation

Checklist Assignment Flow

Admin -> Checklists -> Assign Checklist -> select Employee(s) + Project -> add checklist rows -> choose Frequency + Start Date + Start Time + Attachment Required -> Assign All -> saveChecklistTask() -> CHECKLIST WORK master rows

Checklist Instance Generation Logic

The final production logic should clearly separate CHECKLIST WORK as the master/KRA schedule and CHECKLIST TASK as generated planned work instances. If the current implementation only saves master rows, add a generator function.

Master row -> generator creates instances
 Daily: every day from start date
 Weekly: same weekday
 Fortnightly: every 14 days
 Monthly: same day/date
 Quarterly: every 3 months
 Half Yearly: every 6 months
 Yearly: every 12 months
 One Time: one planned row only

Employee Completion Logic

Employee -> Pending Checklist -> Done -> remarks + attachment if required -> markChecklistTaskDone(taskId, planDate) -> Actual Date -> Delay/On Time Status -> Completed

Bulk Completion Logic

Select checkboxes -> Mark Selected Done -> common remarks/proof -> markChecklistTasksDoneBulk(selectedRows, remarks, filesData) -> update all matched taskId + planDate rows

Checklist Decision

- Option A - Direct mode: Employee Done immediately completes checklist. This is simpler and matches most current UI behavior.
- Option B - Approval mode: Employee Done sends for approval; approver approves/reworks. Use only if checklist proof must be manager verified.
- Do not mix both modes silently. Add explicit setting if needed.

13. Approve/Review Module Logic Documentation

Approve/Review is the command center for manager verification. The frontend merges delegations, work requests and checklists into New Submissions and Rework Submissions tables.

Queue Building Logic

getTasksForApproval(userName, role, filters) -> delegations from team with status Send for Approval + work requests where requester/admin/super admin can approve + checklist approvals if enabled

Approve Action

Click green check -> handleApprovalAction(type, id, Completed, planDate) -> updateStatusWrapper() -> type-specific backend update -> refresh data/cache

Rework Action

Click red redo -> collect rework remarks -> updateStatusWrapper(type, id, Rework, planDate) -> type-specific backend update -> doer sees Rework/Pending again

Proof Columns

- Type: Delegation / Work Request / Checklist.
- From: original assigner/requester/system.
- To: doer who completed the work.
- Task Description: what was done.
- Doer Remarks: completion explanation.
- Proof: uploaded attachments.
- Action: approve/rework buttons.

14. FMS System Logic Documentation

FMS converts a process/flow sheet into actionable rows. A row is pending when Actual Date is blank and completed when Actual Date is filled.

FMS Role Tabs

Role	Tabs
Admin/Super Admin	My Pending Tasks, My Completed Tasks, Team Pending Tasks, Team Completed Tasks
Employee	My Pending FMS, Completed History

FMS Completion Flow

User clicks Done -> markFmsTaskDone(rowId) -> verify row -> if Actual Date empty, set today -> calculate delay vs Planned Date -> write Delay Days + On Time Status -> success

FMS Button Implementation Notes

Button	Production Logic
Import Sheet	Upload/read selected Google Sheet or sheet tab, normalize rows into FMS columns, validate Who/FMS Name/Task Name/Planned Date, clear/reinsert or append by unique key.
Analytics	Show FMS-wise totals, pending, late, bottleneck step, average delay and completion rate.
Generate with AI	Take process prompt/imported headers and draft WHAT/WHEN/HOW/WHO/FMS Name/Task Name/Step/Plan rules for review before saving.
Flow Map	Visualize FMS steps in order with status colors and current bottleneck.

Important FMS Scope Fix

Current recommended behavior: Super Admin sees all FMS rows. Admin should see only rows assigned to self or hierarchy team. Employee sees only rows assigned to self.

15. MIS and Performance Logic Documentation

MIS calculates employee-wise performance across Delegation, Work Request, Checklist and FMS. Screenshots show cards, detailed analysis, average score, total/done/pending/late and KPI drill-down.

MIS Inputs

- Delegation: delegatedTo, taskStatus, targetDate, action/approval date, delay/status, rework remark.
- Work Request: requestFor, status, deadline, completion date, delay/on-time status.
- Checklist: userName, planDate, actualDate, approvalStatus/status, delay/on-time status.
- FMS: who, plannedDate, actualDate, delayDays, onTimeStatus.

MIS Calculated Fields

Field	Logic
Total	All scoped work items for employee/category/filter.
Done	Completed/Approved or Actual Date present.
Pending	Incomplete or Rework awaiting action.
On Time	Completed on or before planned/deadline date.
Late	Completed after due date or overdue pending.
Delay Days	Difference between planned/deadline and actual/today where late.
Reworks	Count of items sent back for rework.
Score	Current UI uses negative penalty style; production formula should be documented and fixed.

Recommended Score Formula

score = 100

score -= pending_rate * 40

score -= late_rate * 30

score -= normalized_delay_days * 20

```
score -= rework_rate * 10
score = clamp(score, 0, 100)
```

Alternative: keep current negative KPI style, but label it "Penalty Score" instead of Avg Score.

MIS Drill-down Logic

Click KPI row -> getDetailedDataForUser(employee, category, filters) -> filter Pending/Overdue/Late based on KPI -> show task ID, description, plan date, actual date, delay, status

Weekly Snapshot Logic

Admin clicks Update -> enter next week target -> saveUserWeeklyScore(employee, target, role) -> getMisData(...) -> append MIS History rows by category/KPI

16. Reports Module Logic Documentation

Report	Source	Important Columns
Performance	getEmployeePerformanceReport	Employee, Total, Completed, Pending, On Time %, Avg Delay, Reworks, Score
Delegation	getDelegationReportData	Task ID, Delegated By, Delegated To, Project, Description, Target Date, Status, Completed Date, Attachments
Work Request	getWorkRequestReportData	Request ID, Requested By, Request For, Project, Description, Deadline, Attachment, Status, Final Remarks, Completed Date
Checklist	getChecklistReportData	Task ID, User, Project, Plan Date, Description, Frequency, Actual Date, Remarks, Attachment
Project	getProjectReportData	Project Name, Total Tasks, Completed, Pending, Delayed, Completion Rate, Delay Rate
FMS	getFmsReportData	FMS Name, Who, Task Name, Planned Date, Actual Date, Delay, Status

Report Filter Rules

- All reports should support employee/team, project, status and date range filters.
- Super Admin report scope = all users. Admin report scope = hierarchy team. Employee report scope = self only if employee report access is enabled.
- Excel/PDF export must export filtered data, not full data unless explicitly requested.

17. Admin Settings Logic Documentation

Manage Users

Add/Edit User -> upsertUser(userData) -> choose Admin Detail if Admin/Super Admin else Employee Detail -> add/update row -> clear user/hierarchy caches

Action	Logic
Add User	Validate user, userId, email, password, role, status; reject duplicate User ID.
Edit User	If role changed, move row to correct sheet.
Delete User	Remove row from Admin Detail/Employee Detail and clear

	caches.
All Roles filter	Client-side filter on user role column.

Manage Projects

Add Project -> manageProject(add) -> append [name, Active]

Power icon -> manageProject(toggleStatus) -> Active <-> Paused

Delete -> manageProject(delete) -> delete project row

- Only Active projects should appear in new task dropdowns.
- Paused projects should remain visible in historical reports.
- Deleting projects should be restricted when tasks already reference that project; prefer Paused instead.

18. Set Hierarchy Logic Documentation

Super Admin -> Add Group -> select Admin + Employees -> saveHierarchy() -> overwrite Hierarchy Setup rows -> clear hierarchy cache

Scenario	Expected Access
Super Admin	All users and all modules.
Admin with group	Self + listed employees.
Admin without group	Self only.
Employee	Self only.

Hierarchy Data Shape

Row format: [AdminName, Employee1, Employee2, Employee3, ...]

Frontend card: Admin header + employee chips + Edit/Delete buttons

19. Notifications Logic Documentation

Sidebar notification dots represent work/action pending for the logged-in user. They are updated after unified app data is loaded.

Notification	Count Logic
Approve/Review dot	approvalCount > 0 from delegation/work-request approvals.
Delegation dot	Pending/Rework delegation tasks assigned to current user.
Checklist dot	Pending checklist instances assigned to current user.
Work Request dot	Pending/Rework work requests assigned to current user as doer.

Login/refresh -> getNotificationCounts(userName, role) -> updateSidebarNotifications(counts) -> hide all dots -> show only non-zero modules

20. State Machines

Delegation

Pending -> Send for Approval -> Completed -> Rework -> Send for Approval -> Completed

Work Request

Pending -> Send for Approval -> Completed -> Rework -> Send for Approval -> Completed

Checklist direct mode

Pending -> Completed / Late / On Time

Checklist approval mode

Pending -> Send for Approval -> Approved -> Rework -> Send for Approval -> Approved

FMS

Pending -> Completed + On Time/Late

Project

Active -> Paused -> Deleted

User

Active -> In-active / Deleted

21. Backend Function Map

Function	Responsibility
checkCredentials	Validate User ID/password and Active status, then return userName and role.
initializeApp	Set frontend currentUser/currentUserRole, show role-based nav, load cached or fresh unified data.
getUnifiedAppData	Return dashboard data, users, projects, notifications and raw module data for fast initial render.
setView	Lazy-load the selected module and call the correct role-specific view loader.
getTeamMembersWithManager	Return all users for Super Admin, current user for Employee, and hierarchy team + admin for Admin.
getNotificationCounts	Count pending work and approval items to show sidebar notification dots.
saveTask	Create delegation rows, save attachments, set Pending, send delegation email.
submitTaskForApproval	Doer completion for delegation: save remarks/proof and set Send for Approval.
updateTaskStatus	Approve/Rework delegation and calculate approval/delay fields.
saveWorkRequest	Create work request rows, safe-parse deadline, save attachments, set Pending, send email.
updateWorkRequestStatus	Doer submits WR for approval; approver completes or reworks.
saveChecklistTask	Save checklist master/KRA rows for selected employees.
markChecklistTaskDone	Complete one checklist instance with actual date/remarks/proof/delay.
markChecklistTasksDoneBulk	Complete multiple checklist instances together.
updateChecklistStatus	Approve or rework checklist when approval mode is used.
getFmsTasksForEmployee	Return FMS rows for admin/all or current employee, with status from Actual Date.
markFmsTaskDone	Set FMS actual date and calculate delay/on-time status.
getMisData / getEmployeePerformanceReport	Calculate employee/category performance metrics and score output.
getFilteredDataForCard / getDetailedDataForUser	Return dashboard/MIS drill-down rows.

getAllReportData	Return report data sets for Work Request, Delegation, Checklist, Project and FMS.
upsertUser / deleteUser	Manage users in Admin Detail or Employee Detail.
manageProject	Add/toggle/delete project in Projects sheet.
saveHierarchy / getHierarchyData	Store and read admin-to-employees mapping.
clearAppCache / getAllAppDataRaw	Central cache invalidation and raw data fetch from transactional sheets.

22. Validation and Security Rules

Frontend Validation

- Required fields: user/project/description/date/time depending module.
- At least one task/request/checklist card must remain.
- File uploads should have size/type validation before base64 conversion.
- Dates should be native date/time fields and converted to ISO-like strings.

Backend Validation

- Check required fields again in Code.gs; never rely only on frontend validation.
- Check logged-in role and hierarchy access before reads/writes.
- Use LockService for write operations that generate IDs or update same rows.
- Use clearAppCache() after every write operation.
- Do not allow editing/deleting users/projects by non-Super Admin.
- Do not expose passwords in user management table or exports.

23. Critical Gaps and Fix Roadmap

Gap	Risk/Impact	Recommended Fix
Branding mismatch	Screenshots show TaskEasy, HTML title/sidebar still says TaskDone.	Rename app title, login branding, sidebar title and email template consistently.
Duplicate functions	Code.gs contains duplicate parseDate_, getChecklistTasksForApproval, getMisData, getMisData_Optimized, getDateRangeYmd and cleanStr. Index contains duplicate formatAttachments/viewAttachments.	Keep one final version of each function; remove older versions to avoid silent override bugs.
Backend role enforcement	Frontend hides modules, but every backend write/read must also verify userRole and team access.	Add requireRole() and requireTeamAccess() checks in save/update/delete functions.
Plain password storage	checkCredentials compares the sheet password directly.	Use hashed passwords or Google identity; do not display passwords in user tables.
Checklist generation ambiguity	saveChecklistTask saves master rows, but generated CHECKLIST TASK instance creation needs to be formally documented and scheduled.	Add scheduled generator or immediate generator for Daily/Weekly/etc. with idempotency.
Checklist Start Time	Frontend has Start Time. Backend must ensure saved start datetime uses this time in plan rows.	Send startDateTime from frontend and parse with parseTaskTargetDateTime_.
FMS admin scope	Current FMS logic treats Admin like Super Admin and may show all rows.	Filter Admin FMS rows by hierarchy team unless explicitly Super Admin.

FMS extra buttons	Import Sheet, Analytics and Generate with AI are visible in screenshot but should have explicit backend handlers.	Implement importFmsSheet, getFmsAnalytics, generateFmsDraft or remove inactive buttons.
Cache invalidation consistency	clearAppCache is called in several save/update functions but should be mandatory for every write.	Add clearAppCache() after every create/update/delete/approve/rework/done operation.
Date standardization	The app already has safe date parsing in key places; enforce one standard everywhere.	Frontend sends YYYY-MM-DDTHH:mm:ss; backend manually parses; display format is separate.

24. Testing / UAT Checklist

Test Case	Expected Result
Login active user	Valid active user logs in and sees correct role navigation.
Login inactive user	Inactive user gets blocked with inactive message.
Super Admin visibility	Super Admin can see all modules and all team data.
Admin hierarchy visibility	Admin sees only self + mapped employees.
Employee visibility	Employee sees only own pending/completed work.
Delegation create	Admin creates multi-doer task; one row per user appears with Pending status.
Delegation done	Doer submits proof; task appears in Approve/Review as Send for Approval.
Delegation rework	Approver sends rework; doer sees task again with remarks.
Work Request create	Requester creates request; doer sees it in Pending For Me.
Work Request approval	Doer submits proof; requester/admin approves and status becomes Completed.
Checklist assign	Admin assigns checklist; planned checklist task appears for employee.
Checklist bulk done	Selected checklist rows complete with same remarks/proof.
FMS done	FMS Actual Date, Delay Days and On Time Status are written correctly.
MIS filter	Last Week/This Month/custom date filters recalculate same employee metrics accurately.
Reports export	Excel/PDF exports match currently visible filtered data.
Cache refresh	After write action, dashboard and notifications show updated counts.

25. Refactor/Migration Recommendations

- Split the app into modules: auth.gs, users.gs, projects.gs, hierarchy.gs, delegation.gs, workRequest.gs, checklist.gs, fms.gs, mis.gs, reports.gs, cache.gs, utils/date.gs, utils/email.gs.
- Create a single constants file for sheet names, column indexes, status values and role values.
- Replace duplicate functions and write unit-like test functions for date parsing, delay calculation and filter ranges.
- Create a central authorize(userName, role, action, targetUser) function.
- Create a Status enum object to avoid spelling mismatch: Pending, Send for Approval, Completed, Rework, Approved, Late, On Time.
- Add a logging sheet for critical changes: who, action, module, rowId, timestamp, oldValue, newValue.
- For long-term scale, migrate to a real backend/database: Node.js/Spring Boot + PostgreSQL/MongoDB + Drive/S3 file storage.

26. Final Master Workflow

Login -> Role Detection -> Dashboard -> Create/Assign Work -> Doer Pending Queue -> Done with Proof -> Approve/Review -> Completed/Rework -> Dashboard + MIS + Reports + Notifications Update

- Every module should support create/assign, pending, done/submit, approval/rework where required, completed history, delay calculation, proof attachment, role visibility, dashboard count, MIS impact and report export.
- The app will feel production-ready when every button has a deterministic backend action, every write clears cache, and every read respects hierarchy.

Appendix A - Screenshot Coverage Used

Screenshot	Observed Module/Logic
admin.png	Admin Settings: Manage Users, roles filter, Add User, Excel/PDF, edit/delete, tabs for projects/settings.
Approve.png	Approve/Review: New Submissions/Rework, From/To filters, Excel/PDF, approve/rework buttons.
Checklist.png	Checklist: Assign Checklist, employee/project selection, frequency, date/time, attachment required, Add Task, Assign All.
Dashborad.png	Dashboard: Team/My toggle, filters, metric cards, project/FMS status, trend chart, personal priority, critical team tasks.
delegation.png	Delegation: Assign Task form, priority, attachment, Add Task, Send All Tasks.
FMS system.png	FMS System: Flow Monitor, Import Sheet, Analytics, Generate with AI, task table, Flow Map/Mark actions.
Mis.png	MIS: date filters, summary cards, card view, employee cards, detailed metrics.
Report.png	Reports: performance cards, detailed performance table, export actions.
set hierarchy.png	Hierarchy: group cards, Add Group, Edit/Delete group, members.
work request.png	Work Request: Create Request, doer/project/description/deadline/attachments, Add Request, Submit All.

Appendix B - Implementation-Ready Logic Prompt

Build/refactor TaskEasy as a role-based workflow management system.

Roles: Super Admin, Admin, Employee.

Data access: Super Admin all data; Admin self + hierarchy employees; Employee self only.

Modules: Dashboard, Delegation, Work Request, Checklist/KRA, Approve/Review, FMS, MIS, Reports, Admin Settings, Set Hierarchy.

Every module must implement: Create/Assign, Pending, Done/Submit, Approval/Rework where required, Completed History, Delay/On Time calculation, Attachments, Role visibility, Dashboard metrics, MIS impact, Report export.

Statuses must be standardized: Pending, Send for Approval, Completed, Rework, Approved, On Time, Late.

Dates must use YYYY-MM-DDTHH:mm:ss internally and must be manually parsed on backend.

All writes must use LockService, validate role/scope, update sheets, clear cache, and return success/error consistently.